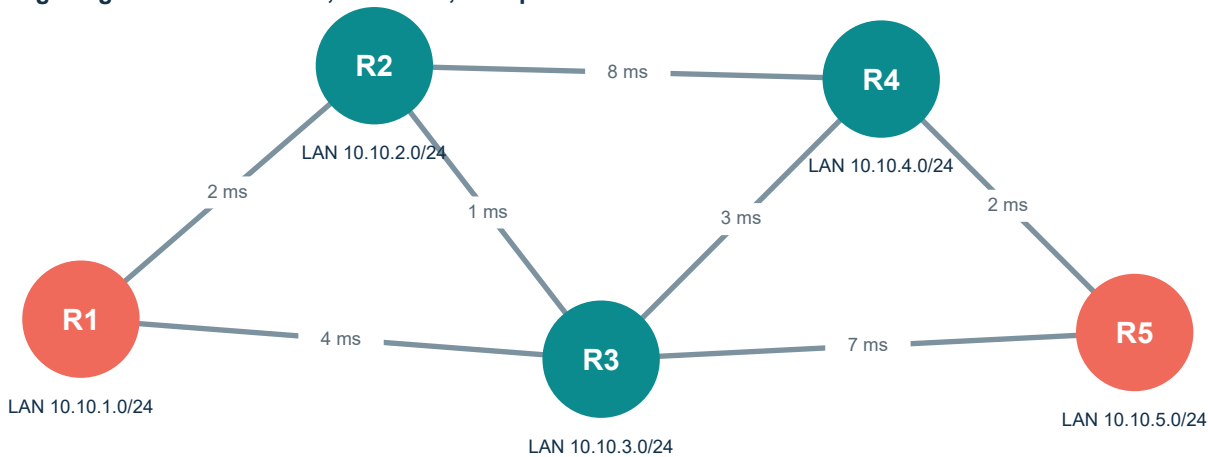


ROTEAMENTO EM MALHA REDUNDANTE

Comparação experimental entre OSPF, eBGP e um algoritmo próprio MC-Dijkstra em cinco roteadores virtuais.

Topologia lógica — 5 roteadores, 7 enlaces, múltiplos caminhos R1 ↔ R5



Plataforma	FRRouting 10.5.1 / Ubuntu 26.04 / VirtualBox
Escopo	5 roteadores • 7 enlaces • 3 soluções isoladas
Data dos ensaios	28–29/09/2026
Entrega do enunciado	01/10/2026 até 19h (Moodle)

1. Resumo executivo

O laboratório foi instalado e executado integralmente em uma VM Ubuntu. A topologia usa cinco namespaces Linux como roteadores, pares veth como enlaces e atrasos artificiais controlados. Foram ensaiados, sem simultaneidade, OSPF, eBGP e o algoritmo próprio MC-Dijkstra.

Resultado principal: as três soluções alcançaram conectividade R1→R5, sustentaram tráfego TCP e recuperaram após a falha de enlace definida para cada cenário.

Solução	Rotas em R1	Controle / 6 s	RTT médio	Taxa TCP	Convergência	Recupero u
OSPF	12	12 pac. / 576 B	19,919 ms	1518,9 Mbit/s	32,000 ms	Sim
eBGP	7	18 pac. / 228 B	24,966 ms	761,9 Mbit/s	262,000 ms	Sim
MC-Dijkstra	7	0 pac. / 0 B	25,552 ms	1233,8 Mbit/s	61,471 ms	Sim

Leitura: OSPF apresentou menor RTT e convergência dinâmica mais rápida; eBGP teve menor taxa nesta amostra e convergiu em 262 ms; o MC-Dijkstra não gera tráfego de controle periódico e recalculou/instalou as rotas em 61,471 ms.

Observação metodológica

Os valores pertencem a uma única VM e medem o comportamento relativo sob a mesma topologia. A taxa TCP pode variar por escalonamento da VM. O volume de controle foi observado por 6 segundos em um enlace de R1; não representa o total agregado da rede.

2. Requisitos e desenho experimental

O enunciado solicita plataforma open source, ao menos cinco roteadores com caminhos redundantes, duas soluções de roteamento configuradas separadamente, algoritmo próprio, mudanças de topologia, métricas e documentação. A tabela abaixo liga cada requisito à evidência produzida.

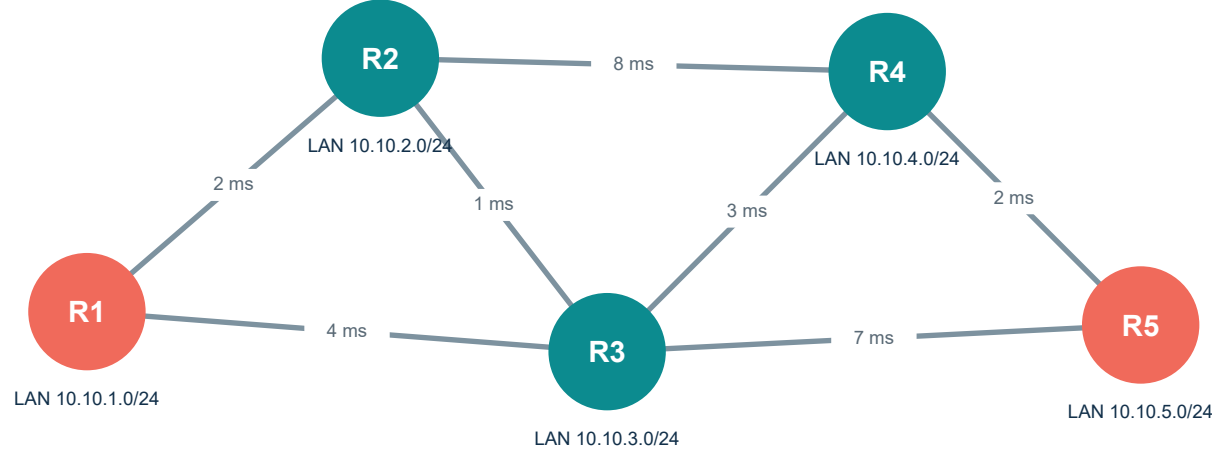
Requisito	Implementação / evidência
Plataforma open source	FRRouting 10.5.1; binários zebra, ospfd e bgpd.
≥ 5 roteadores	R1–R5 em namespaces Linux, cada um com LAN /24.
Redundância	7 enlaces; mais de um caminho entre R1 e R5.
Duas soluções	OSPF e eBGP, executados em rodadas separadas.
Algoritmo próprio	MC-Dijkstra em Python, sem daemon FRR ativo na rodada.
Mudança de topologia	Falha controlada de enlace e verificação de recuperação.
Métricas	Rotas, pacotes/bytes de controle, RTT, TCP e convergência.
Código e evidências	Scripts, configurações, PCAPs, tabelas e logs no pacote ZIP.

Isolamento das rodadas

Antes de cada solução, o script encerra os processos do cenário anterior, remove os namespaces r1–r5 e recria a topologia. Portanto OSPF, eBGP e MC-Dijkstra nunca decidem rotas ao mesmo tempo.

3. Topologia física e lógica

Topologia lógica — 5 roteadores, 7 enlaces, múltiplos caminhos R1 ↔ R5



Enlace	Rede /30	Atraso	Capacidade lógica	Confiabilidade MC
R1–R2	10.0.12.0/30	2 ms	100 Mbit/s	0,99
R1–R3	10.0.13.0/30	4 ms	100 Mbit/s	0,97
R2–R3	10.0.23.0/30	1 ms	50 Mbit/s	0,98
R2–R4	10.0.24.0/30	8 ms	100 Mbit/s	0,98
R3–R4	10.0.34.0/30	3 ms	100 Mbit/s	0,90
R3–R5	10.0.35.0/30	7 ms	100 Mbit/s	0,70
R4–R5	10.0.45.0/30	2 ms	100 Mbit/s	0,99

Cada roteador possui ainda uma interface dummy LAN com endereço 10.10.N.1/24. O tráfego de teste sai de 10.10.1.1 e chega a 10.10.5.1, obrigando o encaminhamento multissalto.

4. Plataforma e automação

A VM recebeu FRROUTING, iperf3, tcpdump e jq. O script principal cria interfaces, aplica atraso com tc netem, gera arquivos por roteador e inicia cada instância FRR em namespace de rede e de montagem isolado.

Comandos de reprodução

```
sudo apt update
sudo apt install -y frr frr-pythontools iperf3 tcpdump jq
chmod +x lab_setup_and_run.sh custom_router.py
sudo ./lab_setup_and_run.sh
```

Estrutura de execução

Etapa	Ação
1	Limpa somente namespaces e sockets do laboratório.
2	Cria R1–R5, LANs, veths e netem.
3	Gera e inicia OSPF; mede e derruba enlace.
4	Recria tudo; gera e inicia eBGP; mede e derruba enlace.
5	Recria tudo; instala rotas MC-Dijkstra; mede e recalcula.
6	Exporta resultados e deixa OSPF ativo para demonstração.

Artefatos gerados: métricas CSV, pings, JSONs iperf3, tabelas por roteador, vizinhanças OSPF, resumo eBGP, rotas antes/depois da falha e capturas PCAP.

5. Solução 1 — OSPF

OSPF é um protocolo de estado de enlace. Cada roteador anuncia sua visão local, todos constroem uma base comum e o SPF escolhe o menor custo. É apropriado para um domínio administrativo e reage rapidamente a falhas.

Configuração principal de R1

```
router ospf
ospf router-id 1.1.1.1
network 10.0.0.0/8 area 0.0.0.0
passive-interface lan
interface r1r2
ip ospf hello-interval 1
ip ospf dead-interval 3
ip ospf network point-to-point
ip ospf cost 2
```

Aspecto	Resultado
Adjacências de R1	R2 e R3 em estado Full
Rotas em R1	12
Controle observado	12 pacotes / 576 bytes em 6 s
RTT R1→R5	19,919 ms
Taxa TCP	1.518,9 Mbit/s
Falha	R3→R4
Recuperação	Sim, 32 ms

Interpretação

O menor RTT e a menor convergência entre os protocolos dinâmicos refletem o conhecimento completo da topologia e o aviso imediato de interface indisponível. A contrapartida é manter adjacências e LSAs.

6. Solução 2 — eBGP

Cada roteador representa um sistema autônomo privado (AS 65001–65005). As sessões eBGP seguem os enlaces físicos e propagam as cinco LANs. A escolha privilegia atributos BGP, especialmente o menor AS_PATH neste laboratório.

Configuração principal de R1

```
router bgp 65001
  bgp router-id 1.1.1.1
  no bgp ebgp-requires-policy
  timers bgp 1 3
  network 10.10.1.0/24
  neighbor 10.0.12.2 remote-as 65002
  neighbor 10.0.13.2 remote-as 65003
```

Aspecto	Resultado
Pares de R1	AS 65002 e AS 65003 estabelecidos
Rotas em R1	7
Controle observado	18 pacotes / 228 bytes em 6 s
RTT R1→R5	24,966 ms
Taxa TCP	761,9 Mbit/s
Falha	R3–R5
Recuperação	Sim, 262 ms

Interpretação

eBGP convergiu mais lentamente e escolheu caminhos por política/AS_PATH, não pelo atraso real. Em troca, oferece excelente separação administrativa e escalabilidade entre domínios.

7. Algoritmo próprio — MC-Dijkstra

O algoritmo modela a rede como grafo não direcionado. Para cada origem executa Dijkstra e instala uma rota estática por LAN remota. O peso combina três dimensões:

$$\text{custo}(e) = \text{atraso_ms} + 100 / \text{capacidade_Mbit/s} + 25 \times (1 - \text{confiabilidade})$$

Termo	Efeito
Atraso	Favorece menor latência acumulada.
100/capacidade	Penaliza enlaces de menor capacidade.
25×(1-confiabilidade)	Evita caminhos historicamente menos confiáveis.
Métrica Linux	10 × custo acumulado, arredondado.

Pseudocódigo

```
para cada origem s:
    dist, primeiro_salto = dijkstra(grafo, s)
    para cada destino d != s:
        limpar rota antiga para LAN[d]
        instalar LAN[d] via primeiro_salto[d]
```

```
na falha R1-R3:
    remover aresta do grafo
    repetir cálculo e instalação
    validar ping R1 -> R5
```

A limpeza explícita do prefixo antes da instalação é essencial: no Linux, métricas diferentes podem coexistir. Sem a limpeza, uma rota antiga de menor métrica poderia manter um black-hole.

8. Resultado do MC-Dijkstra

Aspecto	Resultado
Rotas em R1	7
Controle periódico	0 pacotes / 0 bytes
RTT R1→R5	25,552 ms
Taxa TCP	1.233,8 Mbit/s
Cálculo inicial + instalação	96 ms
Falha simulada	R1–R3
Novo caminho R1→R5	R1–R2–R4–R5
Recomputação + instalação	61,471 ms
Recuperação	Sim

Lógica da escolha

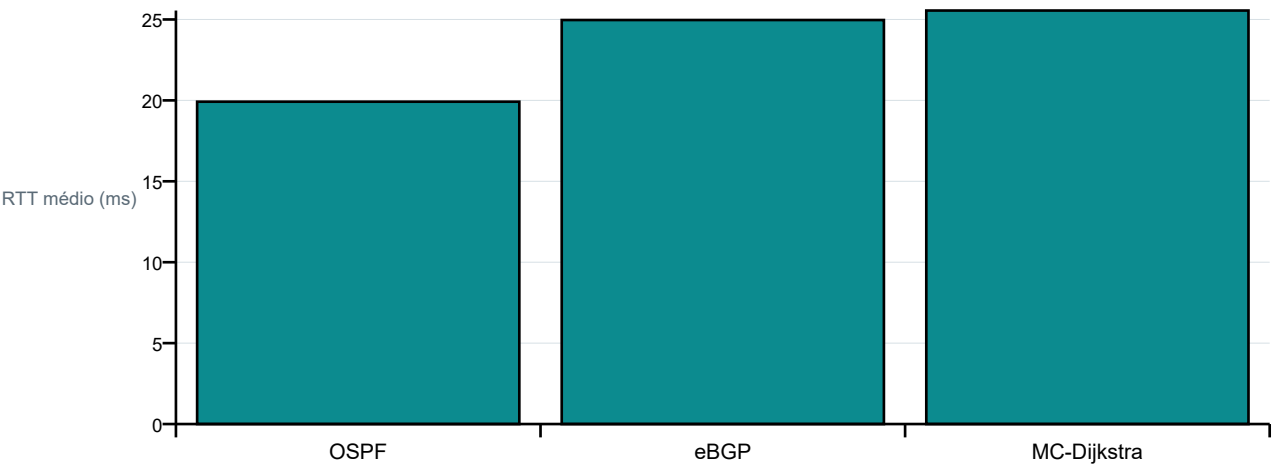
No estado normal, o caminho considera a soma dos pesos e não apenas saltos. Após a remoção de R1–R3, o grafo força o desvio por R2 e R4. As 20 rotas remotas são recalculadas centralmente e instaladas em todos os namespaces.

Limites e evolução

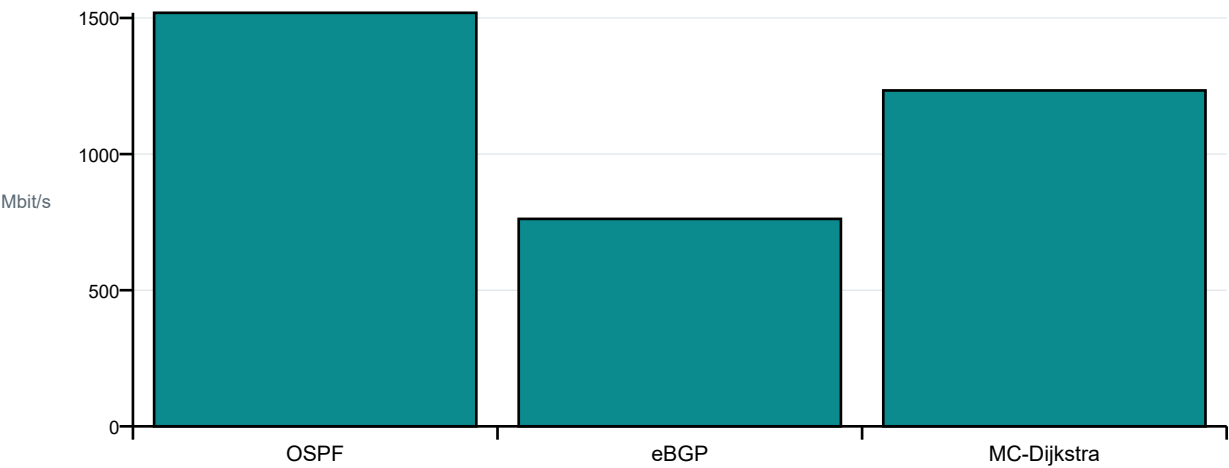
O protótipo conhece a topologia por configuração e exige um gatilho externo de falha. Em produção, seria necessário descobrir vizinhos, autenticar mensagens, distribuir estado e tratar alterações concorrentes. Uma evolução natural seria um controlador SDN ou um daemon com telemetria contínua.

9. Métricas e comparação gráfica

Atraso fim a fim

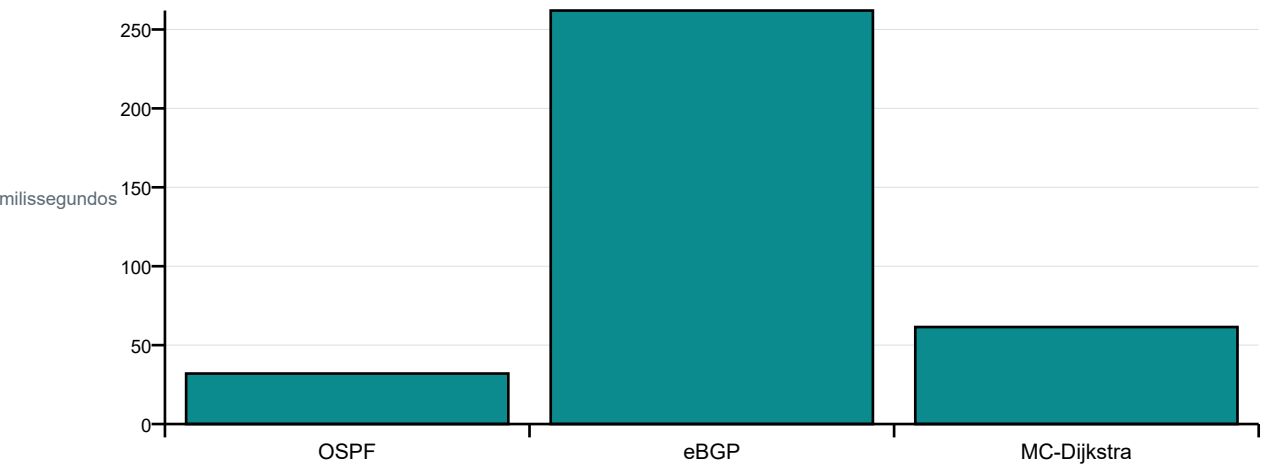


Taxa TCP medida

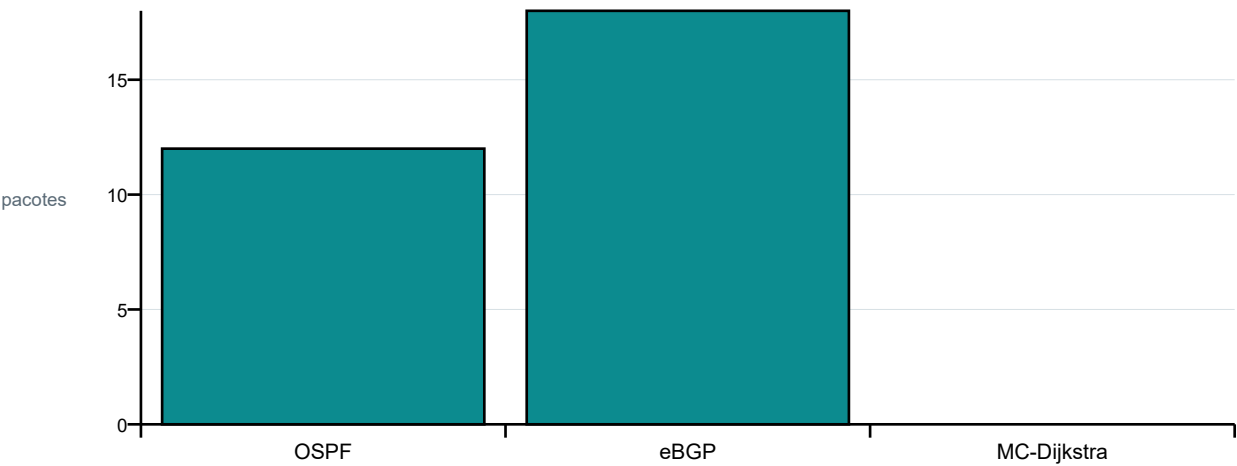


10. Convergência e overhead

Tempo de recuperação / recomputação



Pacotes de controle observados em 6 s



11. Comparação qualitativa

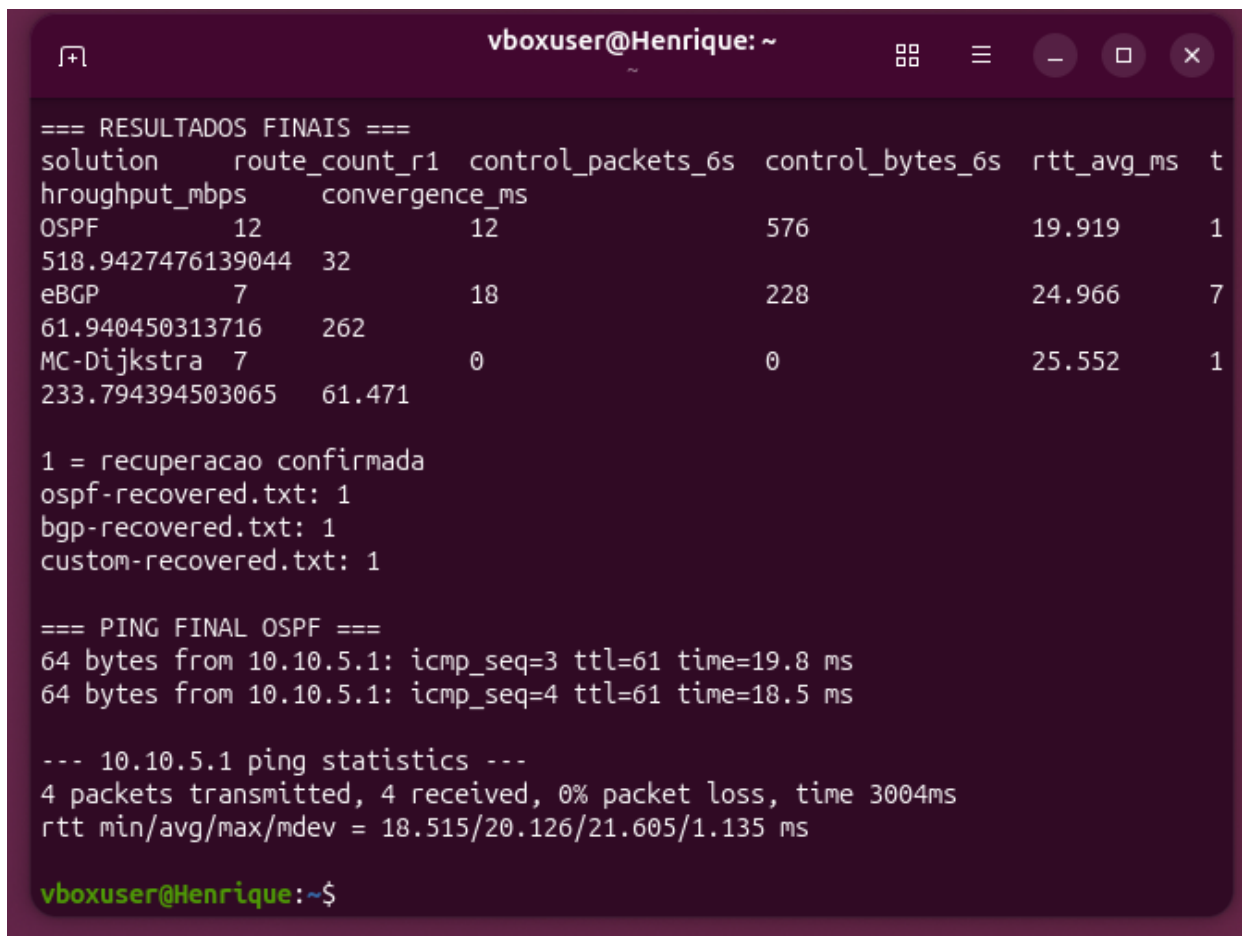
Critério	OSPF	eBGP	MC-Dijkstra
Princípio	Estado de enlace + SPF	Vetor de caminhos + políticas	Grafo ponderado multicritério
Seleção	Menor custo OSPF	Atributos e AS_PATH	Atraso, capacidade, confiabilidade
Mudança de topologia	LSA e novo SPF	Retirada/novo melhor caminho	Gatilho externo + novo Dijkstra
Controle periódico	Hello/LSA	Keepalive/updates	Nenhum no protótipo
Escalabilidade	Boa em áreas internas	Muito alta entre domínios	Limitada pela visão central
Configuração	Moderada	Alta: AS e vizinhos	Código simples; operação ainda manual
Melhor uso	IGP de empresa/campus	Interconexão de domínios	Pesquisa/SDN/otimização específica

Conclusão da comparação

OSPF foi a opção mais equilibrada para esta malha interna. eBGP provou redundância e política, mas seu objetivo natural é interdomínio. O MC-Dijkstra demonstrou que critérios de negócio podem ser incorporados com baixo overhead periódico, embora ainda dependa de controle central e detecção externa.

12. Evidências da VM

A captura abaixo reúne a tabela final de métricas, os três indicadores de recuperação e o ping de demonstração OSPF com 0% de perda.



```
vboxuser@Henrique: ~
=== RESULTADOS FINAIS ===
solution      route_count_r1  control_packets_6s  control_bytes_6s  rtt_avg_ms  t
throughput_mbps  convergence_ms
OSPF           12              12                  576               19.919      1
518.9427476139044 32
eBGP           7              18                  228               24.966      7
61.940450313716 262
MC-Dijkstra    7              0                   0                 25.552      1
233.794394503065 61.471

1 = recuperacao confirmada
ospf-recovered.txt: 1
bgp-recovered.txt: 1
custom-recovered.txt: 1

=== PING FINAL OSPF ===
64 bytes from 10.10.5.1: icmp_seq=3 ttl=61 time=19.8 ms
64 bytes from 10.10.5.1: icmp_seq=4 ttl=61 time=18.5 ms

--- 10.10.5.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 18.515/20.126/21.605/1.135 ms

vboxuser@Henrique:~$
```

Captura realizada diretamente na VM após a rodada corrigida. '1' indica recuperação confirmada.

13. Evidência das adjacências dinâmicas

A VM foi deixada com OSPF ativo para facilitar a demonstração. Antes disso, a rodada eBGP registrou dois pares de R1 estabelecidos (AS 65002 e 65003).

```
vboxuser@Henrique: ~
r2:10.0.12.1      1      0      0
3.3.3.3          1 Full/-    3.945s  2.040s 10.0.13.2      r1
r3:10.0.13.1      1      0      0

=== BGP RESUMO ===

IPv4 Unicast Summary:
BGP router identifier 1.1.1.1, local AS number 65001 VRF default vrf-id 0
BGP table version 5
RIB entries 9, using 1152 bytes of memory
Peers 2, using 47 KiB of memory

Neighbor      V      AS  MsgRcvd  MsgSent  TblVer  InQ OutQ  Up/Down Sta
te/PfxRcd  PfxSnt Desc
10.0.12.2      4      65002    18      20       5    0    0 00:00:11
      4      5 N/A
10.0.13.2      4      65003    19      19       5    0    0 00:00:11
      4      5 N/A

Total number of neighbors 2

=== EXPORTADO ===
vboxuser@Henrique: ~$
```

Os arquivos completos `ospf-neighbors.txt`, `bgp-summary.txt`, tabelas por roteador e PCAPs acompanham o pacote de projeto.

14. Procedimento de demonstração

Passo	Comando / ação	O que provar
1	<code>sudo ./lab_setup_and_run.sh</code>	Execução isolada das três soluções.
2	<code>sudo ip netns exec r1 ping -I 10.10.1.1 10.10.5.1</code>	Conectividade fim a fim.
3	<code>sudo nsenter -t -m -n vtysh</code>	Acesso à instância FRR de R1.
4	<code>show ip ospf neighbor</code>	R2/R3 em Full.
5	<code>show ip route ospf</code>	Rotas aprendidas por OSPF.
6	<code>cat results/metrics.csv</code>	Resultados comparáveis.
7	<code>tcpdump -nn -r pcaps/ospf-control.pcap</code>	Mensagens de controle capturadas.

Para o vídeo, não é necessário executar toda a bateria em tempo real. Pode-se mostrar a execução, cortar os períodos de espera e depois abrir os resultados já gravados, desde que a demonstração inclua um ping e uma falha ao vivo.

Conferência rápida

☐ cinco roteadores visíveis • ☐ dois vizinhos OSPF de R1 • ☐ dois pares eBGP de R1 • ☐ ping R1–R5 • ☐ três linhas no CSV • ☐ recovered=1 nos três cenários

15. Conclusões

O trabalho atingiu o objetivo de comparar decisões e comportamento operacional de soluções distintas sobre a mesma malha. OSPF foi o mais rápido na falha medida (32 ms) e obteve o menor RTT. eBGP recuperou em 262 ms e ilustrou o uso de políticas e AS_PATH. O MC-Dijkstra recuperou em 61,471 ms sem tráfego periódico de controle, incorporando diretamente atraso, capacidade e confiabilidade.

A principal lição é que ‘melhor rota’ depende do objetivo: custo de enlace para OSPF, política para BGP e uma função de utilidade explícita para o algoritmo próprio. A redundância física foi efetivamente utilizada em todos os casos.

Limitações

Os ensaios usam uma única VM e uma amostra por cenário; não representam escala de produção. O MC-Dijkstra é centralizado e o overhead zero corresponde apenas ao protótipo sem mensagens periódicas. Para rigor estatístico adicional, recomenda-se repetir cada teste ao menos 10 vezes e apresentar média, desvio padrão e intervalo de confiança.

Estado final da VM

FRRouting permanece instalado e OSPF está ativo na topologia recriada, permitindo abrir o terminal e demonstrar imediatamente a conectividade e as vizinhanças.

16. O que ainda precisa ser gravado/publicado

A parte técnica está executada e documentada. Restam ações que dependem da identidade do grupo, narração e contas externas.

Item	Ação pendente	Responsável / prazo
Vídeo funcional	Gravar tela + voz; mostrar topologia, vizinhos, ping, falha, recuperação, métricas e MC-Dijkstra.	Grupo • antes de publicar no GitHub
GitHub	Criar/usar repositório, enviar o conteúdo do ZIP, adicionar nomes/matrículas e link/arquivo do vídeo.	Grupo • antes da entrega
Apresentação	Preparar fala de 10 min com divisão: problema, topologia, OSPF, eBGP, algoritmo, resultados e conclusão.	01/10/2026
Moodle	Enviar o material solicitado e confirmar recibo antes das 19h.	01/10/2026 • 19h
Revisão final	Trocar placeholders por nomes, conferir áudio do vídeo e testar links do GitHub.	Antes do envio

Roteiro sugerido para 10 minutos

Tempo	Conteúdo
0:00–1:00	Objetivo e plataforma
1:00–2:30	Topologia e endereçamento
2:30–4:00	OSPF
4:00–5:30	eBGP
5:30–7:30	MC-Dijkstra e fórmula
7:30–9:00	Testes, gráficos e falhas
9:00–10:00	Conclusões e limitações

Arquivos entregues

- `Trabalho\_Roteamento\_FRRouting.pdf`: relatório final.
- `routing-lab-project.zip`: scripts, configurações, resultados, PCAPs, README e checklist de vídeo/entrega.

Datas transcritas do enunciado fornecido: apresentação de 10 minutos em 01/10/2026; GitHub com código/configuração, descrição e vídeo funcional; Moodle até 19h de 01/10/2026.